

Department of Computer and Software Engineering**SE: Machine Learning****Course Instructor:****Date:****Day:****Semester:****Lab 9: Imbalanced Fraud Detection**

Lab Title	CLO	BT	PLO	Weightage
Imbalanced Fraud Detection F1 vs Accuracy & Cost-Sensitive Learning				

Name	Roll Number	Report /10	Viva /5	Total /15

Checked on: _____

Signature: _____

Objectives

- Understand challenges in imbalanced datasets
- Compare Accuracy vs F1-score
- Apply sampling techniques (up/down/SMOTE/SMOTEEN)
- Implement cost-sensitive logistic regression

Problem Statement

You are building a fraud detection system where fraudulent transactions are rare compared to normal ones.

You will analyze how class imbalance affects model performance and explore different techniques to handle imbalance.

Part A: Dataset Exploration

Task 1: Analyze Dataset 1 (60:40)

Load the provided dataset with a 60:40 class distribution (normal vs fraud). Visualize the dataset and observe how the two classes are distributed in the feature space.

Observation: In the 60:40 dataset, the classes are moderately imbalanced. Because their distributions (means and covariances) are relatively close, there is significant overlap in the feature space. However, the model still has enough representation of the minority (fraud) class to attempt establishing a decision boundary without completely ignoring it.

Task 2: Analyze Dataset 2 (90:10)

Load the provided dataset with a higher imbalance (90:10). Visualize and compare it with Dataset 1. Observe how class imbalance affects separability.

Observation: In the 90:10 dataset, the normal class heavily dominates the feature space. The fraud cases are sparse and deeply embedded within the majority class variance. This extreme imbalance destroys linear separability, encouraging standard algorithms to simply predict the majority class to minimize overall error, effectively treating the minority class as negligible noise.

Part B: Logistic Regression and Metrics

Task 3: Train Logistic Regression

Train a logistic regression model using cross-entropy loss:

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

Implementation: A standard Logistic Regression model was trained on the highly imbalanced 90:10 dataset without any class weight adjustments or prior sampling techniques.

Task 4: Evaluate Accuracy and F1 Score

Compute:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$

$$F1 = \frac{2 \cdot (\text{Precision} \cdot \text{Recall})}{\text{Precision} + \text{Recall}}$$

Compare results for both datasets.

Results (90:10 Dataset):

- **Accuracy:** 0.9000
- **F1-Score:** 0.0000

Analysis: The model achieved 90% accuracy simply by predicting the majority class (Normal) for every single transaction. Because it failed to correctly identify a single fraud case (True Positives = 0), both Precision and Recall dropped to 0, resulting in a 0.0000 F1-Score.

Part C: Sampling Techniques

Task 5: Apply SMOTE

Apply: - SMOTE (synthetic minority oversampling)

Observe the effect on model performance.

Results:

- **Accuracy:** 0.5190
- **F1-Score:** 0.1664

Observation: By synthesizing new fraud examples, we forced the model to acknowledge the minority class. This caused a steep drop in raw accuracy (due to increased False Positives), but the F1-Score rose from 0 to 0.1664, meaning the model actually began detecting fraud.

Task 6: Apply SMOTEEN

Apply: - SMOTEEN (SMOTE + cleaning)

Compare results with previous methods.

Results:

- **Accuracy:** 0.4460
- **F1-Score:** 0.1805

Observation: SMOTEENN combines oversampling with undersampling (cleaning noisy boundaries). While the overall accuracy dropped further to 0.4460, the F1-Score improved to 0.1805, outperforming standard SMOTE by creating a cleaner decision boundary between the classes.

Part D: Cost-Sensitive Learning

Task 7: Weighted Loss Function

Modify the loss function to penalize misclassification of fraud more:

$$L = - \sum_{i=1}^N [w_1 y_i \log(p_i) + w_0 (1 - y_i) \log(1 - p_i)]$$

Experiment with different class weights and observe changes.

Results (using balanced weights):

- **Accuracy:** 0.5320
- **F1-Score:** 0.1875

Observation: Cost-sensitive learning yielded the highest F1-Score (0.1875) while maintaining a better accuracy (0.5320) than SMOTEENN. By assigning a heavier mathematical penalty to missed fraud cases during training, the model prioritized the minority class without requiring the fabrication of synthetic data.

Discussion Questions

- **Why can accuracy be misleading in imbalanced datasets?**

Because accuracy measures overall correctness. In a 90:10 dataset, a completely broken model that statically predicts the majority class 100% of the time will achieve 90% accuracy while failing its primary objective: detecting the rare 10% of fraud cases.

- **Which method improved F1-score the most?**

Cost-Sensitive Learning (Weighted Logistic Regression) improved the F1-score the most, reaching 0.1875, compared to SMOTEENN (0.1805), SMOTE (0.1664), and the base model (0.0000).

- **How does cost-sensitive learning compare to sampling methods?**

Sampling methods physically warp the dataset (inventing synthetic points or deleting real ones) before the model trains, which incurs computational costs and can introduce noise. Cost-sensitive learning leaves the dataset untouched and instead modifies the underlying math (the loss function), making a misclassification of the minority class mathematically more expensive to the algorithm. In this experiment, adjusting the math was more effective than faking the data.